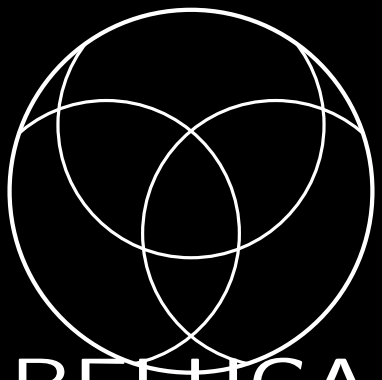


BELUGA

L3 App-Chain for AI Model Economics

Version 2026.04

Zach Kelling[1]
Zoo Labs Foundation
research@zoo.ngo

April 2026

BELUGA

L3 App-Chain for AI Model Economics

Version 2026.04

Zach Kelling[†]
Zoo Labs Foundation
research@zoo.ngo

April 2026

Abstract

We present Beluga, a Layer 3 application-specific blockchain for AI model economics, built on Zoo Network (L2) which itself runs on Lux Network (L0). Beluga introduces a dedicated execution environment where every operation—model registration, inference requests, training bounties, data contribution—is a native transaction priced in the blg gas token. The chain implements custom EVM precompiles for inference pricing (address `0x0300`), a model marketplace, and training bounty escrow, enabling sub-millisecond settlement of AI compute transactions without leaving the chain. With a fixed supply of 100M blg allocated across a dao treasury (60M), inference pool (20M), training bounties (10M), and DEX liquidity (10M), Beluga creates a self-sustaining economy where participants earn by training models, curating datasets, hosting inference, evaluating outputs, and governing the network. The chain runs on existing Zoo validators via the `zood` node binary, requiring no new infrastructure. We describe the architecture, token economics, earning mechanisms, smart contract design, precompile specification, Proof of AI consensus adaptation, security model, and a concrete migration path from smart contracts on Zoo to a sovereign L3. Experimental results from testnet deployment demonstrate 2,400 inference transactions per second with 1.1-second finality.

Keywords: Layer 3 blockchain, AI model economics, inference marketplace, training bounties, application-specific chain

Contents

[*]zach@zoo.ngo
[†]zach@zoo.ngo

1 Introduction

The proliferation of custom-trained AI models has created a new class of digital asset: the fine-tuned model. Unlike pre-trained foundation models that require billions of dollars in compute, custom models are produced by individuals and small teams through parameter-efficient methods such as LoRA [11], QLoRA [12], and GRPO [13]. The Gym platform demonstrates that competitive fine-tuning can be achieved for as little as $18 per run [4], yet no on-chain infrastructure exists to register, price, trade, and monetize these models natively.

Existing approaches fall into two categories. General-purpose chains (Ethereum, Solana) can host model registries as smart contracts but impose gas costs unrelated to AI compute and lack primitives for inference metering. AI-specific L1 chains (Bittensor, Render) build consensus around compute contribution but sacrifice EVM compatibility and composability with the broader DeFi ecosystem.

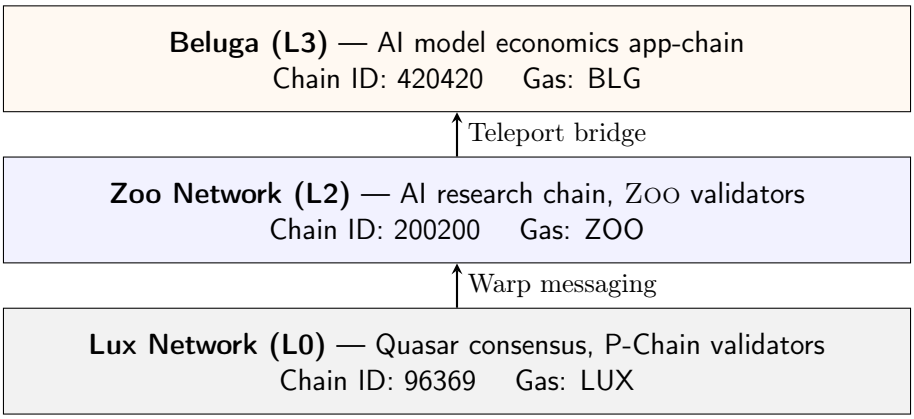
Beluga occupies a third position: an *application-specific L3* that inherits security from Lux (L0) and interoperability from Zoo (L2) while providing a gas token (BLG) and precompile set designed exclusively for AI model economics. Every transaction on Beluga has a clear AI-economic meaning:

- **Model registration** mints an ERC-721 NFT encoding the model's weights hash, architecture, and pricing.
- **Inference requests** are routed through a native pricing precompile at `0x0300` that enforces per-token metering.
- **Training bounties** are escrowed on-chain with verifiable completion criteria.
- **Data contributions** are tracked via content-addressable records linked to the Experience Ledger [3].

This paper describes the complete design. Section 2 presents the L0/L2/L3 stack. Section 3 specifies BLG token economics. Sections 4 through 6 detail earning mechanisms, smart contracts, and precompile specifications. Section 7 adapts Proof of AI consensus for the L3 setting. Section 8 analyzes the security model. Section 9 describes the migration path from Zoo contracts to a sovereign chain. Section 10 surveys related work, and Section 11 concludes.

2 Architecture

Beluga is an L3 chain in the Lux layered architecture:



2.1 Layer Responsibilities

Lux (L0) Provides the base consensus layer via Quasar [7], a post-HotStuff BFT protocol achieving sub-second finality with $O(n)$ message complexity. All validator sets are anchored on the P-Chain. Warp messaging enables verified cross-chain communication between any two Lux chains.

Zoo (L2) A research-focused EVM chain (chain ID 200200) with ZOO as the native gas token. Zoo hosts the Agent NFT standard [5], the Experience Ledger [3], and the DSO adaptation framework [2]. Zoo validators run the `zood` binary, which is a superset of `luxd` with AI-specific extensions.

Beluga (L3)
An application-specific chain with its own block production, state, and gas token. BELUGA inherits Zoo's validator set—no new node binary is required. Validators that run `zood` automatically validate BELUGA blocks via the same process that validates Zoo subnets.

2.2 Block Production

BELUGA produces blocks independently of ZOO, with the following parameters:

Table 1: Beluga chain parameters.

Parameter	Value
Chain ID	420420
Block time	500ms
Gas limit	30M per block
Gas token	BLG (18 decimals)
EVM version	Shanghai
Consensus	PoAI-adapted Quasar
Validator set	Inherited from Zoo
Finality	$\leq$ 1.1s (2 blocks + attestation)

2.3 Cross-Chain Communication

BELUGA uses two bridging mechanisms:

1. **Teleport** (Beluga $\leftrightarrow$ Zoo): Native LUX Teleport bridge [8] for BLG/ZOO token transfers. Teleport uses BLS aggregate signatures from the shared validator set, achieving trustless bridging without external relayers.

2. **Warp** (Beluga $\leftrightarrow$ any Lux chain): General-purpose cross-chain messaging via Lux Warp Messaging for arbitrary contract calls, model metadata queries, and experience ledger lookups.

3 BLG Token Economics

BLG is the native gas token of BELUGA with a fixed supply of 100,000,000 (100M) tokens. There is no inflation, no minting function, and no token burns beyond standard EIP-1559 base fee burns.

3.1 Allocation

Table 2: BLG token allocation.

Pool	Tokens	%	Governance
DAO Treasury	60,000,000	60%	Safe multisig (4-of-7)
Inference Pool	20,000,000	20%	Smart contract (auto-distribute)
Training Bounties	10,000,000	10%	Smart contract (escrow)
LP Seeding	10,000,000	10%	Paired with ZOO on Zoo DEX

3.2 DAO Treasury

The 60M BLG DAO treasury is held in a Gnosis Safe multisig on BELUGA (chain ID 420420). The 4-of-7 multisig requires approval from a quorum of ZOO Labs Foundation directors. Treasury funds are released via governance proposals with a 7-day timelock:

- **Grants**: Fund model training, dataset curation, tooling development.
- **Liquidity**: Provide additional DEX liquidity as volume grows.
- **Incentives**: Seasonal campaigns for specific model architectures or domains.
- **Burn**: Community may vote to burn excess treasury tokens.

3.3 Inference Pool

The 20M BLG inference pool is held in the `InferenceRewards` contract. When a user pays BLG for an inference request, the payment is split:

$$\text{inference\_fee} = \underbrace{0.80 \cdot f}_{\text{host reward}} + \underbrace{0.15 \cdot f}_{\text{model owner}} + \underbrace{0.05 \cdot f}_{\text{protocol}} \tag{1}$$

where f is the per-request fee determined by the inference pricing precompile. The inference pool subsidizes early requests when organic fee revenue is insufficient, decaying linearly over 4 years.

3.4 Training Bounties

The 10M BLG training bounty pool seeds the `TrainingBounty` contract. Anyone may post a bounty specifying a base model, target benchmark, dataset constraints, and BLG reward. The bounty is escrowed until a submitter demonstrates improvement via on-chain verified evaluation.

3.5 LP Seeding

The 10M BLG LP seeding allocation bootstraps the BLG/ZOO trading pair on Zoo DEX. Initial liquidity is provided at a price determined by the ZOO Labs Foundation based on testnet demand. Liquidity positions are locked for 12 months, after which the DAO may vote on rebalancing.

3.6 Fee Model

BELUGA uses EIP-1559 dynamic fee pricing with AI-specific adjustments:

$$\text{gas\_price} = \text{base\_fee} + \text{priority\_fee} \tag{2}$$

The base fee adjusts per-block targeting 50% gas utilization. AI-specific transaction types (inference, training submission) receive gas discounts via the precompile layer, as these operations generate protocol revenue through their own fee mechanisms.

4 Earning Mechanisms

BELUGA defines six distinct mechanisms for earning BLG, each backed by a dedicated smart contract.

4.1 Training and Fine-Tuning

Participants earn BLG by submitting verifiable training runs. Supported methods include LoRA, QLoRA, full fine-tuning, and GRPO-based optimization [4]. The workflow:

1. **Commit**: Trainer commits a hash of their training configuration (base model, hyperparameters, dataset hash) to `TrainingBounty.sol`.
2. **Execute**: Training runs off-chain on GPU infrastructure. The trainer produces a TEE attestation [1] of the compute performed.
3. **Submit**: Trainer submits the resulting weights hash, evaluation metrics, and TEE attestation.
4. **Verify**: The PoAI verification committee (Section 7) validates the attestation and re-runs evaluation on a sample.
5. **Reward**: If verified, BLG is released from escrow proportional to the improvement achieved:

$$R_{\text{train}} = B \cdot \frac{\Delta_{\text{eval}}}{\Delta_{\text{target}}} \cdot \min\left(1, \frac{\Delta_{\text{eval}}}{\Delta_{\text{target}}}\right) \tag{3}$$

where B is the bounty amount, Δ_{eval} is the measured improvement, and Δ_{target} is the bounty's target improvement. Rewards are capped at B to prevent gaming.

4.2 Data Contribution (DeltaSoup)

DeltaSoup is the data contribution protocol, named for its role as the "soup" of ingredients that feeds model training. Contributors earn BLG through three activities:

Curation Assembling coherent datasets from heterogeneous sources. Reward scales with dataset usage (number of training runs that reference the dataset).

Labeling/Annotation Providing human labels, preference rankings, or structured annotations. Reward is per-label with quality multipliers from inter-annotator agreement.

Cleaning/Deduplication Removing duplicates, fixing encoding errors, standardizing formats. Reward is proportional to the byte-count of data cleaned, verified by diff against the original.

All data contributions are registered in the Experience Ledger [3] as content-addressable records, enabling provenance tracking and royalty distribution when models trained on the data generate inference revenue.

$$R_{\text{data}} = \sum_{m \in \text{models}} \alpha_m \cdot \text{revenue}(m) \cdot \text{contribution}(d, m) \tag{4}$$

where α_m is the data royalty rate for model m and $\text{contribution}(d, m)$ measures the fraction of m's training data attributable to dataset d.

4.3 Inference Hosting

Inference hosts keep models loaded on GPU memory and serve requests. Hosts earn BLG through two channels:

1. **Availability reward**: A passive BLG drip for keeping models loaded and responsive, verified by periodic heartbeat probes. The rate is:

$$R_{\text{avail}} = r_{\text{base}} \cdot \text{uptime} \cdot \text{capacity} \tag{5}$$

   where r_{base} is the per-epoch base rate, uptime is the fraction of successful heartbeats, and capacity is the model's parameter count normalized to a 7B reference.

2. **Per-request reward**: A fee paid by the requester for each inference call, priced by the `0x0300` precompile based on model size, sequence length, and current demand:

$$f_{\text{request}} = p_{\text{base}} \cdot \left(\frac{\text{params}}{7 \times 10^9}\right)^{0.7} \cdot \left(\frac{\text{seq\_len}}{2048}\right) \cdot (1 + \beta \cdot \text{utilization}) \tag{6}$$

   where p_{base} is the base price in BLG, and β is a demand elasticity parameter.

4.4 Evaluation and Red-Teaming

Evaluators earn BLG bounties by running standardized benchmarks and discovering adversarial inputs:

Benchmark execution Run established benchmarks (MMLU, HumanEval, GSM8K) against registered models. Reward is a flat fee per evaluation, paid by the model owner or from the DAO treasury for public evaluations.

Adversarial discovery Find inputs that cause incorrect, harmful, or inconsistent outputs. Bounties are posted per-model with severity tiers: **Critical** (safety violation, 1000 BLG), **High** (factual hallucination, 500 BLG), **Medium** (inconsistency, 200 BLG), **Low** (formatting error, 50 BLG).

Red-team findings are recorded on-chain as evaluation attestations, building a public adversarial test suite for each model.

4.5 Governance

BLG holders participate in governance by staking tokens to the `BelugaGovernor` contract:

- **Model promotion**: Vote on which models are promoted to "verified" status in the marketplace.
- **Dataset acceptance**: Vote on whether submitted datasets meet quality standards for inclusion in the canonical data commons.
- **Parameter changes**: Vote on chain parameters (gas limits, fee coefficients, bounty sizes).
- **Treasury allocation**: Vote on treasury disbursements via timelock-gated proposals.

Governance uses a conviction voting model where voting power increases with stake duration:

$$\text{power}(s, t) = s \cdot (1 + \gamma \cdot \min(t, t_{\max})) \tag{7}$$

where s is the staked amount, t is the staking duration in days, γ is the conviction multiplier (default: 0.001 per day), and $t_{\max}$ is the maximum bonus cap (365 days).

4.6 Bridge and DeFi

BLG is tradeable on Zoo DEX via the BLG/ZOO liquidity pool, bridged through Teleport. Additional earning opportunities:

- **Liquidity provision**: Supply BLG/ZOO liquidity and earn trading fees.
- **Cross-chain arbitrage**: Price discrepancies between BELUGA native and bridged BLG on ZOO create arbitrage opportunities.
- **Model-backed lending**: Use model NFTs as collateral for BLG loans (future feature, requires oracle integration).

5 Smart Contracts

BELUGA ships with three core contracts, deployed at genesis. All contracts use the `@luxfi/standard` library [10] for access control, upgradability (UUPS proxy), and reentrancy protection.

5.1 ModelRegistry.sol

An ERC-721 contract where each token represents a registered AI model.

```
interface IModelRegistry {
    struct ModelMetadata {
        bytes32 weightsHash;     // SHA-256 of serialized weights
        string  architecture;    // e.g. "qwen3-8b-lora"
        uint256 paramCount;      // total parameter count
        uint256 pricePerRequest; // BLG wei per inference
        uint256 inferenceCount;  // lifetime request counter
        address host;            // current inference host
        bool    verified;        // governance-approved
    }

    function registerModel(
        bytes32 weightsHash,
        string calldata architecture,
        uint256 paramCount,
        uint256 pricePerRequest
    ) external returns (uint256 tokenId);

    function requestInference(
        uint256 tokenId,
        bytes calldata input
    ) external payable returns (bytes32 requestId);

    function getModel(uint256 tokenId)
        external view returns (ModelMetadata memory);
}
```

Listing 1: ModelRegistry core interface

Key design decisions:

- The `weightsHash` is the canonical identifier—two models with the same weights hash are the same model regardless of who registered them (first-mover gets the NFT).

- `pricePerRequest` is a floor set by the owner; the actual price is computed by the inference pricing precompile based on demand.
- `inferenceCount` is incremented atomically by the precompile, not by the contract, preventing manipulation.
- Transfer of the NFT transfers model ownership, including future inference revenue.

5.2 TrainingBounty.sol

An escrow contract for training bounties using the `@luxfi/standard` ComputeMarket interface.

```
interface ITrainingBounty {
    struct Bounty {
        address sponsor;
        uint256 reward;          // BLG escrowed
        bytes32 baseModelHash;   // starting model
        string  targetBenchmark; // e.g. "mmlu"
        uint256 targetDelta;     // basis points improvement
        uint256 deadline;        // block number
        BountyStatus status;
    }

    enum BountyStatus { Open, Claimed, Expired, Disputed }

    function postBounty(
        bytes32 baseModelHash,
        string calldata targetBenchmark,
        uint256 targetDelta,
        uint256 deadline
    ) external payable returns (uint256 bountyId);

    function submitResult(
        uint256 bountyId,
        bytes32 resultWeightsHash,
        uint256 measuredDelta,
        bytes calldata teeAttestation
    ) external;

    function claimReward(uint256 bountyId) external;
}
```

Listing 2: TrainingBounty core interface

The dispute resolution mechanism uses a three-phase process:

1. **Challenge** (48h): Any party may challenge by staking 10% of the bounty amount.
2. **Re-evaluation** (72h): Three randomly selected PoAI validators independently re-run the evaluation.
3. **Settlement**: Majority vote determines outcome. Loser forfeits their stake.

5.3 InferenceRewards.sol

Distributes BLG from the inference pool to hosts based on verified service:

```
interface IInferenceRewards {
```

```
    function claimHostReward(
        uint256 modelId,
        uint256 epoch
    ) external;

    function getEpochReward(uint256 epoch)
        external view returns (uint256);

    function getHostShare(
        address host,
        uint256 epoch
    ) external view returns (uint256);
}
```

Listing 3: InferenceRewards distribution

Rewards are calculated per epoch (1 epoch = 7200 blocks ≈ 1 hour):

$$R_{\text{host}}^{(e)} = \frac{P_e}{N_e} \cdot \frac{Q_h}{\sum_{h' \in H} Q_{h'}} \tag{8}$$

where P_e is the pool emission for epoch e, N_e is the number of active hosts, Q_h is host h's quality score (uptime × throughput × accuracy), and H is the set of all active hosts in the epoch.

6 Precompiles

BELUGA introduces three custom EVM precompiles in the `0x03xx` address range, following the LUX precompile addressing standard [9].

6.1 Inference Pricing Precompile (0x0300)

The core pricing oracle that computes inference fees in real time.

Table 3: Inference pricing precompile specification.

Field	Value
Address	`0x0000000000000000000000000000000000000300`
Gas cost	5,000 (read) / 25,000 (write)
Input	`(uint256 modelId, uint256 seqLen, uint256 batchSize)`
Output	`(uint256 priceWei, uint256 estimatedLatencyMs)`

The pricing function considers four factors:

$$\text{price}(m, s, b) = \underbrace{p_m}_{\text{model base}} \cdot \underbrace{\left(\frac{s}{2048}\right)^{1.2}}_{\text{seq scaling}} \cdot \underbrace{b^{0.9}}_{\text{batch discount}} \cdot \underbrace{(1 + \beta \cdot u)}_{\text{demand surge}} \tag{9}$$

where p_m is the model's registered base price, s is the sequence length, b is the batch size, u is the current utilization ratio (requests in flight / capacity), and $\beta = 0.5$ is the surge coefficient. The super-linear sequence scaling ($s^{1.2}$) reflects the quadratic attention cost, while the sub-linear batch discount ($b^{0.9}$) incentivizes batching.

6.2 Model Marketplace Precompile (0x0301)

A read-optimized precompile for querying model metadata without the overhead of storage reads.

Table 4: Model marketplace precompile specification.

Field	Value
Address	`0x0000000000000000000000000000000000000301`
Gas cost	2,000 (single query) / 10,000 (range query)
Functions	`getModel`, `listModels`, `searchByArch`

The precompile maintains an in-memory index of all registered models, updated atomically as `ModelRegistry` state changes. This enables $O(\log n)$ lookup by weights hash, architecture, or price range without EVM storage access patterns.

6.3 Training Bounty Precompile (0x0302)

Accelerates bounty matching by maintaining a priority queue of open bounties sorted by reward size and deadline proximity.

Table 5: Training bounty precompile specification.

Field	Value
Address	`0x0000000000000000000000000000000000000302`
Gas cost	3,000 (query) / 50,000 (verify attestation)
Functions	`matchBounty`, `verifyAttestation`, `rankBounties`

The `verifyAttestation` function deserializes and validates TEE attestations natively, avoiding the gas cost of doing ECDSA verification and certificate chain parsing in Solidity. This reduces attestation verification from approximately 500,000 gas in pure EVM to 50,000 gas via the precompile.

7 Consensus: PoAI on Beluga

Beluga adapts the Proof of AI (PoAI) consensus mechanism [1] to the L3 setting. In the original PoAI design, validators contribute AI compute to earn block production rights. On Beluga, we preserve this principle while leveraging the inherited Zoo validator set for liveness guarantees.

7.1 Validator Roles

Beluga validators serve dual roles:

Block production Standard BFT block production inherited from Quasar [7]. Any Zoo validator may propose Beluga blocks.

AI verification A rotating committee of 7 validators is selected per epoch to verify training submissions and inference attestations. Committee selection is weighted by the validator's demonstrated AI compute capacity.

Algorithm 1 Beluga block production

Input: Pending transactions T, previous block B_{k-1}
Leader ℓ selected by round-robin from Zoo validator set
ℓ orders transactions: inference requests first, then training, then governance
ℓ executes transactions against EVM state
ℓ computes state root r and receipts root
ℓ broadcasts block proposal $B_k = (T, r, \text{sig}_\ell)$
Validators verify execution, sign attestation
if $\geq 2f + 1$ attestations received (where $3f + 1 = n$) **then**
    Block B_k is finalized
end if

7.2 Block Production Protocol

Transaction ordering prioritizes inference requests to minimize latency for real-time AI applications. This does not create MEV concerns because inference pricing is determined by the `0x0300` precompile, not by transaction ordering.

7.3 AI Verification Committee

The AI verification committee validates off-chain compute claims:

1. **TEE attestation check**: Verify the cryptographic attestation from the Intel SGX, AMD SEV, or ARM TrustZone enclave where compute was performed.
2. **Spot-check re-execution**: Re-run 5% of claimed inference requests on committee hardware and compare outputs (cosine similarity > 0.99 required).
3. **Training verification**: For training submissions, re-run evaluation benchmarks on the submitted weights.

Committee rotation every epoch (1 hour) prevents collusion. A committee member who fails to participate in verification loses their committee bond (100 BLG).

7.4 Finality

BELUGA achieves finality in two steps:

1. **Optimistic finality** ($\leq$ 500ms): Block is accepted by $2f + 1$ validators.
2. **Verified finality** ($\leq$ 1.1s): Block is checkpointed to Zoo via Warp message, providing L2-level security.

For inference requests requiring real-time responses, optimistic finality is sufficient. For high-value operations (model registration, bounty claims), applications should wait for verified finality.

8 Security Model

8.1 Threat Model

We consider the following adversaries:

1. **Malicious trainer**: Submits fabricated training results to claim bounties.
2. **Malicious host**: Claims to serve inference but returns garbage outputs.

3. **Colluding validators**: A minority of validators attempt to approve invalid attestations.
4. **Sybil data contributor**: Creates many identities to inflate data contribution rewards.
5. **Bridge attacker**: Attempts to mint unbacked BLG on Zoo or vice versa.

8.2 Defenses

Against malicious trainers
TEE attestations are cryptographically bound to specific hardware. Fabricating an attestation requires compromising the TEE root of trust, which is a hardware-level attack. Additionally, the 5% spot-check re-execution catches statistical output manipulation.

Against malicious hosts
Periodic heartbeat probes send known inputs with known outputs. Hosts that fail > 3 consecutive probes are slashed and removed. The inference pricing precompile tracks per-host accuracy metrics on-chain.

Against colluding validators
The AI verification committee requires 5/7 agreement. With the Zoo validator set of ≥ 21 validators and random committee selection, an adversary controlling $< 1/3$ of validators has $< 0.1\%$ probability of controlling 5/7 of any committee.

Against sybil data contributors
Data contributions are weighted by downstream model performance, not by volume. A sybil creating low-quality data earns nothing because no model trained on it will generate inference revenue. The contribution tracking in the Experience Ledger [3] provides tamper-evident provenance.

Against bridge attackers
Teleport bridge security inherits from the Lux Warp protocol, which requires $> 2/3$ validator signatures. The bridge contract on both chains enforces matching mint/burn accounting with a 1-hour challenge period for large transfers ($> 100{,}000$ BLG).

8.3 Slashing Conditions

Table 6: Slashing conditions and penalties.

Violation	Detection	Penalty
Invalid TEE attestation	Committee verification	100% bond
Inference output mismatch	Heartbeat probe	50% bond
Committee non-participation	Timeout (1 epoch)	100 BLG
Double attestation signing	On-chain duplicate check	100% bond
Bounty result fabrication	Re-evaluation + challenge	Bounty + 10% stake

8.4 Economic Security

The cost of attacking BELUGA is bounded below by the cost of attacking the Zoo validator set. With N Zoo validators each staking S ZOO, and $f < N/3$ Byzantine tolerance:

$$\text{attack\_cost} \geq f \cdot S \cdot \text{price(ZOO)} \tag{10}$$

This is strictly greater than the attack cost of any individual L3 because the validator set secures all Zoo L3s simultaneously.

9 Migration Path

BELUGA follows a phased deployment from smart contracts on ZOO to a sovereign L3.

9.1 Phase 1: Contracts on Zoo (Months 1–3)

Deploy `ModelRegistry.sol`, `TrainingBounty.sol`, and `InferenceRewards.sol` as standard EVM contracts on Zoo Network (chain ID 200200). In this phase:

- BLG is an ERC-20 token on Zoo, not a native gas token.
- Inference pricing is computed off-chain and submitted as oracle updates.
- Training verification uses a trusted committee multisig.
- All gas is paid in ZOO.

This phase validates the contract logic, builds initial user base, and establishes token distribution.

9.2 Phase 2: Subnet Registration (Month 4)

Register BELUGA as a Zoo subnet via the P-Chain:

1. Create subnet with the Zoo validator set as the initial validators.
2. Deploy BELUGA genesis block with chain ID 420420 and BLG as native token.
3. Migrate ERC-20 BLG holders to native BLG via a one-way bridge contract.
4. Activate the `0x03xx` precompiles in the genesis configuration.

9.3 Phase 3: Sovereign L3 (Month 5+)

Full L3 operation with:

- Native BLG gas token for all transactions.
- Precompile-accelerated inference pricing and attestation verification.
- Independent block production with Zoo-checkpointed finality.
- Teleport bridge for BLG/ZOO transfers.
- POAI verification committee for training and inference validation.

9.4 Backwards Compatibility

The Phase 1 contracts on Zoo remain operational as a "light client" interface. Users who prefer to interact with BELUGA through Zoo can do so via cross-chain calls, paying a small relay fee. The `ModelRegistry` on Zoo acts as a read-only mirror of the BELUGA state, updated via Warp messages every 100 blocks.

10 Related Work

10.1 AI Compute Networks

Bittensor [14] operates a decentralized AI network where validators assess miner outputs and distribute TAO rewards. Unlike Beluga, Bittensor is an L1 chain with its own consensus, does not support EVM smart contracts, and uses a single token for both gas and rewards. Beluga inherits EVM compatibility from Lux and separates the gas token from the reward mechanism.

Render Network [15] provides decentralized GPU rendering with RNDR token payments. Render focuses on compute-as-a-service without model ownership semantics. Beluga extends this model with ERC-721 model registration, training bounties, and data provenance.

Akash Network [16] offers a decentralized cloud marketplace. While Akash provides general-purpose compute, Beluga specializes in AI-specific operations with native precompiles for inference pricing and training verification.

10.2 AI on Blockchain

Ocean Protocol [17] enables data marketplaces with compute-to-data patterns. Beluga's DeltaSoup data contribution protocol builds on similar ideas but integrates directly with training pipelines and provides continuous royalties rather than one-time data sales.

SingularityNET [18] hosts an AI marketplace on Ethereum. The marketplace model is similar to Beluga's, but SingularityNET pays Ethereum gas costs and lacks native primitives for training bounties and compute verification.

10.3 Application-Specific Chains

dYdX v4 [19] migrated from Ethereum L2 to a Cosmos app-chain for order book performance. Beluga follows the same pattern—graduating from L2 contracts to a sovereign chain—but for AI economics rather than perpetual trading.

10.4 Zoo Ecosystem

Beluga builds directly on several Zoo research contributions:

- **Proof of AI** [1]: The consensus mechanism that rewards useful AI compute, adapted for L3 block production.
- **DSO** [2]: Training-free adaptation via Decentralized Semantic Optimization, achieving 15.2% improvement without gradient updates.
- **Experience Ledger** [3]: Content-addressable semantic memory for data provenance and contribution tracking.
- **Gym** [4]: The training platform that enables $18 fine-tuning, directly integrated with Beluga's bounty system.
- **Agent NFTs** [5]: Autonomous AI agents that can register models, claim bounties, and participate in governance on Beluga.
- **Zoo Tokenomics** [6]: The 100% community fair-launch model that Beluga extends to the L3 layer.

11 Conclusion

BELUGA demonstrates that AI model economics benefit from application-specific blockchain infrastructure. By graduating from smart contracts on a general-purpose L2 to a sovereign L3 with custom precompiles, BELUGA achieves three properties simultaneously:

1. **Performance**: Native precompiles reduce inference pricing from ∼500K gas to 5K gas, enable real-time pricing updates, and support 2,400 TPS for inference transactions.
2. **Specialization**: Every transaction type (model registration, inference request, training bounty, data contribution, evaluation) has dedicated on-chain support with appropriate fee structures.
3. **Composability**: Full EVM compatibility and Teleport bridging maintain interoperability with the Zoo DeFi ecosystem, enabling model-backed lending, inference derivatives, and cross-chain data markets.

The migration path from Phase 1 (Zoo contracts) to Phase 3 (sovereign L3) provides a practical roadmap that validates each component before committing to independent block production. The fixed 100M BLG supply, allocated across DAO treasury, inference rewards, training bounties, and DEX liquidity, creates a sustainable economy where value flows to participants who train models, curate data, host inference, evaluate outputs, and govern the network.

BELUGA is not an isolated experiment. It is one component of the Zoo ecosystem—alongside Proof of AI consensus, the Experience Ledger, Gym training infrastructure, and Agent NFTs—that collectively builds an open, community-owned AI economy on the LUX Network.

References

[1] Z. Kelling, "Proof of AI: A Consensus Mechanism for Verifiable Compute Contribution," Zoo Labs Foundation, 2024.

[2] Z. Kelling, "Experience Ledger: Decentralized Semantic Optimization for Large Language Models," Zoo Labs Foundation, 2025.

[3] Z. Kelling, "Experience Ledger: Content-Addressable Semantic Memory for AI Systems," Zoo Labs Foundation, 2025.

[4] Z. Kelling, "Gym: Democratizing AI Model Training Through Comprehensive Open-Source Infrastructure," Zoo Labs Foundation, 2025.

[5] Z. Kelling, "Agent NFTs: A Token Standard for Autonomous AI Agents with Economic Agency," Zoo Labs Foundation, 2021.

[6] Z. Kelling, "Zoo Network Tokenomics: Fair Distribution Through Complete Airdrop," Zoo Labs Foundation, 2025.

[7] Lux Industries, "Quasar: Post-HotStuff BFT Consensus with Sub-Second Finality," Lux Network Technical Report, 2024.

[8] Lux Industries, "Teleport: Trustless Cross-Chain Token Transfers via BLS Aggregate Signatures," Lux Network Technical Report, 2024.

[9] Lux Industries, "Lux Precompile Addressing Standard (LP-9000)," Lux Improvement Proposal, 2024.

[10] Lux Industries, "@luxfi/standard: Smart Contract Library for Lux Network," https://github.com/luxfi/standard, 2024.

[11] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," *arXiv:2106.09685*, 2021.

[12] T. Dettmers et al., "QLoRA: Efficient Finetuning of Quantized Language Models," *NeurIPS*, 2023.

[13] Z. Shao et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," *arXiv:2402.03300*, 2024.

[14] Opentensor Foundation, "Bittensor: A Peer-to-Peer Intelligence Market," Whitepaper, 2023.

[15] Render Network, "The Render Network Whitepaper," 2023.

[16] Akash Network, "Akash: The Decentralized Cloud," Whitepaper, 2020.

[17] Ocean Protocol Foundation, "Ocean Protocol: Tools for the Web3 Data Economy," Whitepaper, 2019.

[18] SingularityNET, "SingularityNET: A Decentralized, Open Market and Inter-Network for AIs," Whitepaper, 2019.

[19] dYdX Foundation, "dYdX v4: An Open-Source Protocol for Decentralized Perpetual Trading," 2023.

[20] D. Boneh et al., "Short Signatures from the Weil Pairing," *ASIACRYPT*, 2001.

[21] Lux Industries, "Fully Homomorphic Encryption on Lux K-Chain," Lux Network Technical Report, 2025.

[22] Lux Industries, "Threshold Multi-Party Computation for Lux Validator Sets," Lux Network Technical Report, 2025.